

Cramer’s Rule, Matrices, and Their Place in Data Science and Machine Learning

Aksel Aghajanyan

April 6, 2026

Abstract

Cramer’s rule gives an explicit formula for the solution of a nonsingular square linear system in terms of determinants. This note reviews the rule, places it in the computational landscape of numerical linear algebra, and surveys applications in data science and machine learning: from regression, graphical models, and generative modeling to control, reinforcement learning, geometric computer vision, and optimization. We emphasize when closed-form determinant reasoning clarifies structure, when small systems admit hand analysis, and why production pipelines rely on factorizations and iterative solvers instead of Cramer’s rule at scale.

1 Introduction

Many problems in data science and machine learning reduce to linear algebra: least squares, projections, eigendecompositions, and linear systems $A\mathbf{x} = \mathbf{b}$. For a square matrix $A \in \mathbb{R}^{n \times n}$, *Cramer’s rule* expresses each component of the unique solution as a ratio of determinants. The result is elegant and historically important, but its role in modern computation is nuanced: it is invaluable for theory, pedagogy, and tiny structured problems, while standard ML software favors factorizations and iterative methods for speed and stability.

2 Cramer’s rule

Let A be invertible and $\mathbf{b} \in \mathbb{R}^n$. Denote by $A^{(i)}$ the matrix obtained by replacing the i -th column of A with $\mathbf{b}$.

Theorem 1 (Cramer’s rule). *If $\det(A) \neq 0$, then the unique solution to $A\mathbf{x} = \mathbf{b}$ satisfies*

$$x_i = \frac{\det(A^{(i)})}{\det(A)}, \quad i = 1, \dots, n. \quad (1)$$

Equivalently, using the adjugate (classical adjoint) $\text{adj}(A)$,

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A), \quad (2)$$

which makes the connection between Cramer’s rule and the explicit inverse explicit.

3 Computational perspective

3.1 Cost and stability

Computing $\det(A)$ and n modified determinants via expansion or naive formulas grows very quickly with n . Practical dense linear solvers use LU decomposition, Cholesky (for positive

definite A), or QR methods, typically $\mathcal{O}(n^3)$ with better constants and pivoting for numerical stability. For sparse or structured matrices, iterative methods (conjugate gradients, Krylov subspace methods) scale with problem structure.

Therefore, **Cramer’s rule is not used to solve large linear systems** in production ML stacks (NumPy, PyTorch, JAX, etc.). The rule remains a *conceptual* tool: it shows that solvability is equivalent to $\det(A) \neq 0$ and that each x_i depends rationally on the entries of A and $\mathbf{b}$.

3.2 When explicit determinant ratios still matter

- **Low-dimensional analysis.** In two or three variables, hand calculation or symbolic algebra (SymPy, Mathematica) can use Cramer’s rule for exact rational answers.
- **Teaching.** It reinforces determinants, column operations, and invertibility.
- **Theory.** Proofs in optimization and geometry sometimes use $\det(A) \neq 0$ or explicit minors; Cramer’s rule is one way to package that logic.

4 Applications in data science and machine learning

The following applications illustrate how *square linear systems* and *determinants* arise throughout DS/ML. Cramer’s rule is rarely the implementation choice when n is large, but the same objects— $\det(A)$, minors, and explicit solution structure—often appear in models, objectives, and proofs.

4.1 Linear regression, ridge, and elastic net

The ordinary least squares (OLS) problem $\min_{\beta} \|X\beta - \mathbf{y}\|_2^2$ for full-column-rank X yields the normal equations $X^\top X\beta = X^\top \mathbf{y}$. Cramer’s rule applies in principle when $X^\top X$ is $p \times p$ and invertible, giving each β_i as a ratio of determinants—useful for $p = 2, 3$ in exercises and for *sensitivity* intuition (how β_i changes when a column of X or an entry of $\mathbf{y}$ moves). Production code uses QR or SVD on X for stability. Ridge regression,

$$(X^\top X + \lambda I)\beta = X^\top \mathbf{y},$$

is a nonsingular system for $\lambda > 0$; again, Cholesky or factorization is standard. For generalized linear models, iteratively reweighted least squares reduces each outer step to a weighted least-squares linear system.

4.2 Gaussian models, precision matrices, and log-likelihoods

For a multivariate Gaussian $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$, the density contains $\det(\Sigma)^{-1/2}$ and the quadratic form $\mathbf{x}^\top \Sigma^{-1} \mathbf{x}$. In Gaussian graphical models, zeros in the precision matrix Σ^{-1} encode conditional independences. Evaluating training objectives (log marginal likelihood), computing conditional distributions, and Kalman-style updates all require solving linear systems and sometimes $\log \det(\Sigma)$ —determinant-related quantities tied to “volume” in feature space. Gaussian processes reuse the same pattern: covariance matrices define posteriors over functions, and $\log \det(K)$ appears in the marginal likelihood.

4.3 Probabilistic graphical models and counting

The matrix-tree theorem and variants express spanning-tree counts and partition-function pieces as determinants of Laplacian minors. Determinantal point processes (DPPs) build diversity-promoting sampling using minors of a kernel matrix. Here determinants measure *dependence*

volume among items; Cramer’s rule is not the computational workhorse, but minor/determinant language is native to the model class.

4.4 Generative flows, VAEs, and change of variables

Normalizing flows and invertible networks apply a change of variables: the log-density adds $\log |\det J|$ where J is the Jacobian of the bijective map. Invertibility and local volume scaling are exactly determinant-level concepts. Training maximizes likelihood through stable Jacobian log-determinant estimates (often via specialized parameterizations), not by expanding Cramer’s rule for huge n .

4.5 Computer vision and geometry

Estimating a planar homography or fundamental matrix from correspondences leads to small linear systems (often refined by SVD for noise). For 3×3 templates, one can relate unknown parameters to determinant ratios in principle; in practice, SVD and RANSAC dominate pipelines. Bundle adjustment and structure-from-motion repeatedly solve sparse linearized systems (Levenberg–Marquardt), illustrating the gap between *small symbolic* linear algebra and *large sparse* numerical solves.

4.6 Optimization: Newton and interior-point methods

A Newton step for smooth minimization solves $H\mathbf{d} = -\mathbf{g}$ with Hessian H and gradient $\mathbf{g}$. Trust-region and line-search variants rely on accurate, stable solutions of symmetric systems. Interior-point methods for convex programs (linear, quadratic, SDP) solve large KKT systems each iteration. Cramer’s rule is irrelevant at scale, but the idea that a unique step exists when H (or the KKT matrix) is nonsingular is the same invertibility story as $\det(A) \neq 0$.

4.7 Control, filtering, and dynamical systems

Linear state-space models, Kalman filters, and LQR/LQG controllers update estimates via Riccati equations and structured linear solves. Low-dimensional state spaces (n small) appear in robotics and tracking; teaching and prototyping sometimes use explicit small-matrix inverses. At scale, factorizations exploit sparsity and structure.

4.8 Reinforcement learning and policy evaluation

Evaluating a fixed policy in a finite Markov decision process can be written as linear systems involving the transition matrix and reward vector (e.g. Bellman equations for value functions). For tabular problems the dimension is the number of states; direct solves or iterative policy evaluation replace explicit Cramer formulas. Deep RL still uses linear algebra inside optimizers and second-order approximations.

4.9 Kernel machines and support vectors

Kernel ridge regression leads to an $n \times n$ linear system $(K + \lambda I)\boldsymbol{\alpha} = \mathbf{y}$. Support vector machines (in their primal/dual formulations) involve quadratic programs whose KKT conditions are linear complementarity problems, not a single Cramer step—but low-dimensional toy SVMs in courses sometimes reduce to small linear systems where explicit formulas echo Cramer’s logic.

4.10 Recommendation systems and matrix factorization

Alternating least squares for matrix completion repeatedly solves least-squares problems for user and item latent vectors. Each subproblem is a modest k -dimensional normal equation; closed-form inverse or Cholesky is used when k is small. The overall problem is large-scale and iterative.

4.11 Differentiable programming and implicit layers

Implicit layers and deep equilibrium models define hidden states $\mathbf{z}^*$ as the solution of $F(\mathbf{z}, \mathbf{x}) = \mathbf{0}$ or a fixed point. Backpropagation uses implicit differentiation through the Jacobian of F at $\mathbf{z}^*$, which reduces to solving another linear system. Automatic differentiation couples with conjugate gradients or GMRES; symbolic Cramer’s rule is not used for large Jacobians.

4.12 Summary of the pattern

Across these areas, **linear systems and determinants are everywhere; Cramer’s rule as an algorithm is for small n , teaching, or symbolic work.** Recognizing both keeps models interpretable while guiding efficient implementation.

5 Summary

Cramer’s rule ties the solution of $\mathbf{Ax} = \mathbf{b}$ to matrix determinants and illuminates invertibility and rational dependence of $\mathbf{x}$ on $(\mathbf{A}, \mathbf{b})$. Data science and machine learning repeatedly instantiate the same ingredients—normal equations, covariances and precisions, Jacobians in flows, KKT and Hessian systems, Bellman linearizations, and graph Laplacian minors—while production code chooses factorizations and iterative solvers for scale and stability. Keeping both views—explicit structure (including Cramer’s rule when n is tiny) and engineered numerical linear algebra—supports sound modeling and efficient systems.

References

- [1] G. Strang, *Linear Algebra and Learning from Data*, Wellesley-Cambridge Press, 2019.
- [2] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
- [3] S. Boyd and L. Vandenberghe, *Introduction to Applied Linear Algebra*, Cambridge University Press, 2018.